Assignment 3: EC5644

Ronan Engel

13 November 2026

1 Question 1

1.1 Dataset

This problem relies upon four separate Gaussians with unique means and covariances and equal priors. It is critical there is some degree of overlap, resulting in the below parameters for the Gaussians.

$$\mu_0 = \begin{bmatrix} 2 \\ 1.5 \\ 1 \end{bmatrix}, \quad \mu_1 = \begin{bmatrix} 1 \\ -1 \\ -1.5 \end{bmatrix}, \quad \mu_2 = \begin{bmatrix} -2 \\ 2 \\ -2 \end{bmatrix}, \quad \mu_3 = \begin{bmatrix} -2 \\ -1 \\ 2 \end{bmatrix}$$

$$\Sigma_0 = \begin{bmatrix} 2 & -1.5 & 0.5 \\ -1.5 & 4 & 1 \\ 0.5 & 1 & 10 \end{bmatrix}, \quad \Sigma_1 = \begin{bmatrix} 2 & -0.2 & -0.5 \\ -0.2 & 4 & 0.5 \\ -0.5 & 0.5 & 3 \end{bmatrix}, \quad \Sigma_2 = \begin{bmatrix} 2 & 0.1 & 0.5 \\ 0.1 & 3 & 1 \\ 0.5 & 1 & 3 \end{bmatrix}, \quad \Sigma_3 = \begin{bmatrix} 4 & 0.1 & 0.5 \\ 0.1 & 4 & 0.1 \\ 0.5 & 0.1 & 1 \end{bmatrix}$$

Through these Gaussian parameters, samples of size 100, 500, 1,000, 5,000, 10,000, and 100,000, making the set $\mathcal{D} \in \{ \mathcal{D}_{train}^{100}, \mathcal{D}_{train}^{500}, \mathcal{D}_{train}^{1000}, \mathcal{D}_{train}^{5000}, \mathcal{D}_{train}^{10k}, \mathcal{D}_{test}^{100k} \}$. The true labels for $\mathcal{D}_{test}^{100k}$ depicted in Figure 1.

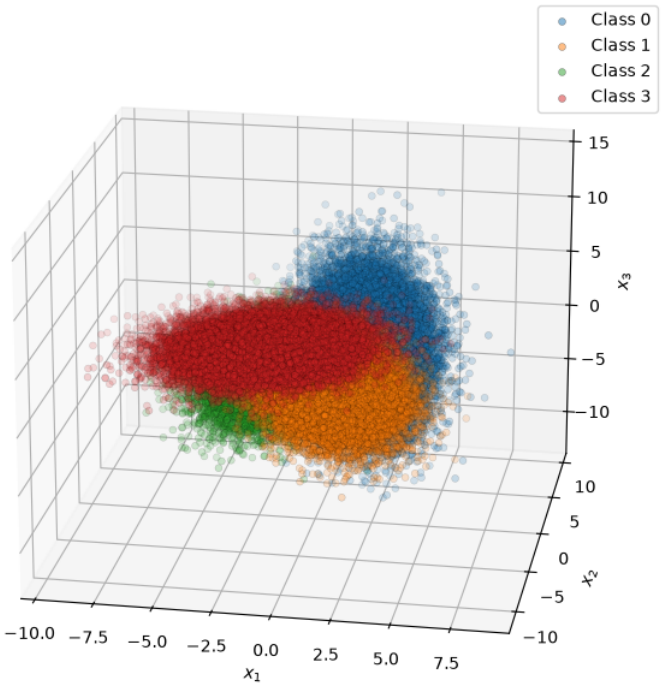


Figure 1: $\mathcal{D}_{test}^{100k}$ Visualization

1.2 Theoretically Optimal Classifier

The best possible classifier can be derived through a Maximum A Posteriori (MAP) decision rule. The MAP rule is defined in Equation 1, which defines the rule and then applies Bayes Rule for Equation 2.

$$D(\mathbf{x}) = \underset{c \in \{1,\ldots,C\}}{\arg\max}\ P(L = c \mid \mathbf{x}) \tag{1}$$

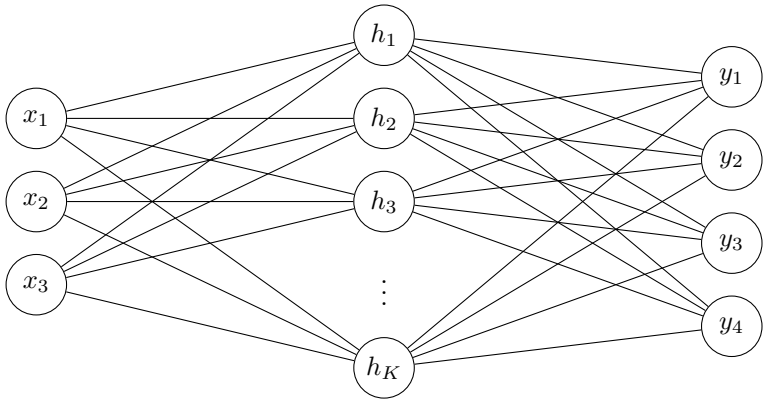
$$D(\mathbf{x}) = \underset{c \in \{1,\ldots,C\}}{\arg\max}\ p(\mathbf{x} \mid L = c)\, P(L = c) \tag{2}$$

From Equation 2, the logarithm can be applied, leaving us with the below Equation 3. This decision rule can the be applied to all samples within the $\mathcal{D}_{test}^{100k}$, predictions on each of the data sets labels. These labels yield an error of .1786 and serves as the optimal MAP classification error.

$$D(\mathbf{x}) = \underset{c \in \{1,\ldots,C\}}{\arg\max}\ \left[\ln p(\mathbf{x} \mid L = c) + \ln P(L = c)\right] \tag{3}$$

1.3 Multi Layer Perceptron Training Pipeline

For this Classification problem, we will utilize a Multi Layer Perceptron as a classifier for the datasets defined in Section 1.1. The network structure will be a two layer multi layer perceptron with one hidden layer of P neurons in $P \in \{1, 2, 4, 8, 16, 32, 64\}$, with a model being trained for each hidden layer width and the best structure being chosen for each dataset in $\mathcal{D}$. The network utilizes an Exponential Linear Unit, ELU, activation function, for the hidden layer and a softmax for each class in the final output layer.



Input layer	Hidden layer	Output layer
$\mathbf{x} \in R^3$	P neurons, ELU	4 neurons, softmax

For each of the datasets, $\mathcal{D}$, a ten fold cross validation was conducted for each of the hidden layer widths, P, with the average cross validation error being derived from testing on the validation set for each width. The optimal perceptron width was decided for each of the datasets based on this minimized cross validation error, P'_N, which is then utilized to train the model again, with 5 random restarts with randomly initialized weights to ensure a local minimum isn't impeding optimal model performance. Finally, this final model is applied to the $\mathcal{D}_{test}^{100k}$ to derive the test error for the given model. These results are relayed in Table 1.

$\mathcal{D}_{train}^{N}$	P'	CV error	Test error
$\mathcal{D}_{train}^{100}$	32	0.2200	0.2086
$\mathcal{D}_{train}^{500}$	8	0.2500	0.1885
$\mathcal{D}_{train}^{1000}$	32	0.1760	0.1834
$\mathcal{D}_{train}^{5000}$	16	0.1876	0.1825
$\mathcal{D}_{train}^{10k}$	64	0.1772	0.1804

1.4 Results

Based upon these results, there are clear patterns that emerge that regarding error and perceptron widths. Although expanding the width of the hidden layer incurs additional computational and time costs in training, Figure 3 shows that additional complexity minimizes error, to a certain extent.

To ensure the software tools performed the intended computations, several checks were made. The per-class sample means and covariances of the generated data were compared against the specified parameters, confirming that labels were aligned with their samples. Training used Keras's `sparse_categorical_crossentropy` loss with a softmax output, which equals the average negative log-likelihood of the labels, so minimizing it is maximum likelihood estimation; labels were numbered 0–3 to match the four outputs. Test decisions take the argmax of the network's output posteriors, which is the MAP rule under 0-1 loss. Cross-validation used `StratifiedKFold` on the training set only, with a freshly initialized model for every fold, so the test set was never used for model selection. Finally, every trained MLP achieved a test error at or above the theoretically optimal error of 0.1786, as theory requires.

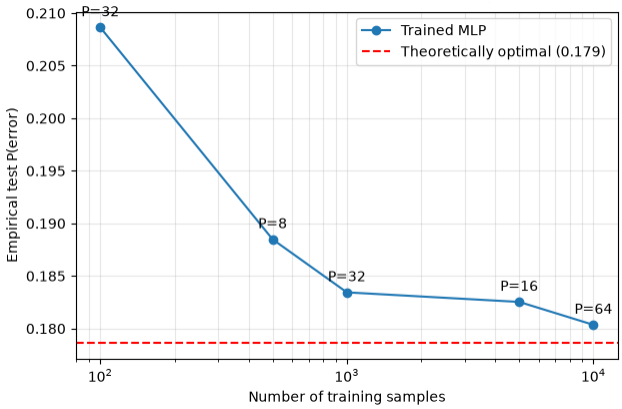


Figure 2: Test Error versus Training Set Size

Additionally, these gains in accuracy across the models is seen across all datasets, with model order driving down error until the becoming generally stagnant around $P = 8$. This performance is seen in Figure 3.In conclusion, without prior knowledge of the distribution, and with significant enough data and neuron complexity, MLP can arrive close to MAP estimation with no prior knowledge of the distribution.

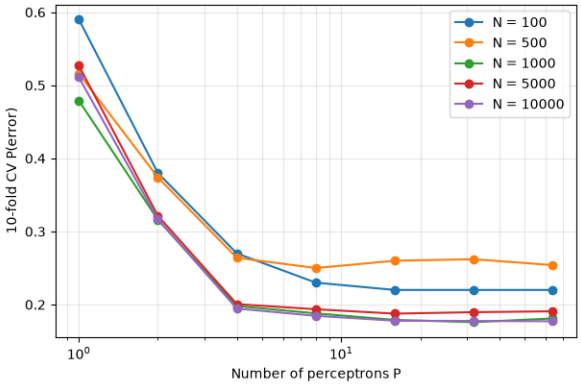


Figure 3: Model Order Selection over Datasets

2 Question 2

2.1 Parameter Creation

This problem establishes four Gaussian distributions with the caveat that two of the distributions must have a difference of means equal to their sum of average covariance matrix eigenvalues. This caveat is applied to μ_0 and μ_1, creating the parameters described below. Additionally, the priors for these distributions are described in $P(x)_k \in \{0.4, 0.3, 0.2, 0.1\}$.

$$\mu_0 = \begin{bmatrix} 0 \\ -4.5 \end{bmatrix}, \quad \mu_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mu_2 = \begin{bmatrix} -4 \\ 5 \end{bmatrix}, \quad \mu_3 = \begin{bmatrix} 7 \\ 0 \end{bmatrix}$$

$$\Sigma_0 = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 2 \end{bmatrix}, \quad \Sigma_1 = \begin{bmatrix} 2.5 & 1 \\ 1 & 2.5 \end{bmatrix}, \quad \Sigma_2 = \begin{bmatrix} 4 & 1.5 \\ 1.5 & 2 \end{bmatrix}, \quad \Sigma_3 = \begin{bmatrix} 2 & 1.1 \\ 1.1 & 5 \end{bmatrix}$$

These parameters were utilized to create datasets with sample size 10, 100, and 1000, denoted as $\mathcal{D}_{10}$, $\mathcal{D}_{100}$, and $\mathcal{D}_{1000}$. A visualization of the Gaussians are depicted in Figure 4, with a sample size of 10,000.

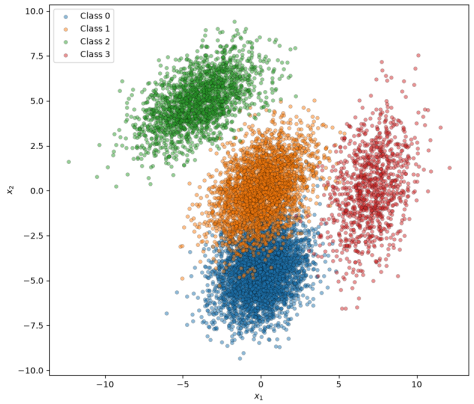


Figure 4: $\mathcal{D}$ parameters visualized at $N = 10,000$

2.2 Gaussian Mixture Model and the EM Algorithm

A Gaussian mixture model (GMM) represents a probability density as a weighted sum of K Gaussian components:

$$p(\mathbf{x} \mid \theta) = \sum_{k=1}^{K} \alpha_k \mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_k, \Sigma_k), \qquad \alpha_k \geq 0, \quad \sum_{k=1}^{K} \alpha_k = 1,$$

with parameters $\theta = \{\alpha_k, \boldsymbol{\mu}_k, \Sigma_k\}_{k=1}^{K}$. Each sample can be viewed as generated in two steps: a component $z \in \{1, \ldots, K\}$ is drawn with probability $P(z = k) = \alpha_k$, and then $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_z, \Sigma_z)$. The component label z is latent, meaning it is not observed in the data.

The parameters are estimated by maximum likelihood, maximizing

$$\ln L(\theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \alpha_k \mathcal{N}(\mathbf{x}_i;\, \boldsymbol{\mu}_k, \Sigma_k).$$

Because the logarithm acts on a sum, this has no closed-form maximizer. The Expectation-Maximization (EM) algorithm solves it iteratively. Starting from an initial $\theta^{(0)}$, it alternates two steps.

E-step. Using the current parameters, compute the responsibility of each component for each sample, the posterior probability that sample i came from component k:

$$\gamma_{ik} = \frac{\alpha_k \mathcal{N}(\mathbf{x}_i;\, \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^{K} \alpha_j \mathcal{N}(\mathbf{x}_i;\, \boldsymbol{\mu}_j, \Sigma_j)}.$$

M-step. With $N_k = \sum_i \gamma_{ik}$, update each component as a responsibility-weighted Gaussian fit:

$$\alpha_k = \frac{N_k}{N}, \qquad \boldsymbol{\mu}_k = \frac{1}{N_k}\sum_{i=1}^{N} \gamma_{ik}\,\mathbf{x}_i, \qquad \Sigma_k = \frac{1}{N_k}\sum_{i=1}^{N} \gamma_{ik}\,(\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top.$$

Each iteration is guaranteed not to decrease $\ln L(\theta)$, and the steps repeat until the improvement falls below a tolerance. EM converges to a local maximum that depends on $\theta^{(0)}$, so each fit is run from 10 different k-means-based initializations, keeping the solution with the highest log-likelihood. A small ridge, $\Sigma_k + \lambda I$, is added to each covariance to keep it invertible. These steps are implemented by scikit-learn's `GaussianMixture`.

2.3 Results

For each datasets, the 10 fold cross validation was conducted, with candidate solutions of $K \in \{1, 2, 3, ..., 10\}$ clusters applied to the data. This through the expectation maximization steps outlined in above, the best fit of clusters, K, was found for each distribution. This process of cross validation and refitting was repeated 100 times, utilizing a new seed to initialize the parameters of the dataset to give slightly different samples, with the average K, μ_K, and Σ_K for the given datasets for the mode K value.

2.3.1 $\mathcal{D}_{10}$

As visualized below in Figure 5, $\mathcal{D}_{10}$ is an incredibly spares dataset that looks far from the traditional gaussian model. Due to the sparseness, the algorithm struggled to find much resembling clusters and gaussian, resulting in an optimal $K = 1.03$ as the minimizer of log odds for the distributions. Derived parameters are below, as well as a visualization of the sample and these parameters sampled at $N = 10,000$.

$$\hat{\boldsymbol{\mu}}_0 = \begin{bmatrix} -0.14 \\ -0.76 \end{bmatrix}, \quad \hat{\Sigma}_0 = \begin{bmatrix} 9.71 & -3.34 \\ -3.34 & 13.95 \end{bmatrix}, \quad \hat{\alpha}_0 = 1$$

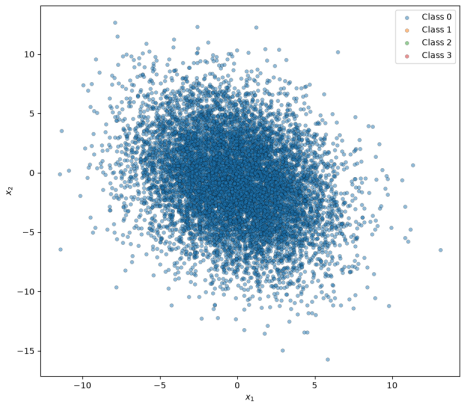


Figure 5: $\mathcal{D}^{10}$ Estimated Parameters Visualized at $N = 10000$

2.3.2 $\mathcal{D}_{100}$

As visualized below in Figure 6, $\mathcal{D}_{100}$ begins to approximate closed to the true distributions of the dataset. An optimal $K = 3.34$ is derived, the sparseness had the classifier struggled to see true class $\{0, 1\}$ from one another. This is evidenced both visually and the through the combination of the true priors of $\{0, 1\}$, being combined into one prior, $0.4 + 0.3 \approx 0.62$ The average derived parameters and distribution is modeled below, sampled at $N = 10,000$.

$$\hat{\boldsymbol{\mu}}_0 = \begin{bmatrix} 4.92 \\ 0.25 \end{bmatrix}, \quad \hat{\boldsymbol{\mu}}_1 = \begin{bmatrix} -0.08 \\ -3.14 \end{bmatrix}, \quad \hat{\boldsymbol{\mu}}_2 = \begin{bmatrix} -4.00 \\ 5.05 \end{bmatrix}$$

$$\hat{\Sigma}_0 = \begin{bmatrix} 6.11 & 0.35 \\ 0.35 & 4.68 \end{bmatrix}, \quad \hat{\Sigma}_1 = \begin{bmatrix} 1.88 & 0.43 \\ 0.43 & 5.46 \end{bmatrix}, \quad \hat{\Sigma}_2 = \begin{bmatrix} 3.70 & 1.45 \\ 1.45 & 2.03 \end{bmatrix}$$

$$\hat{\boldsymbol{\alpha}} = \begin{bmatrix} 0.19 & 0.62 & 0.19 \end{bmatrix}$$

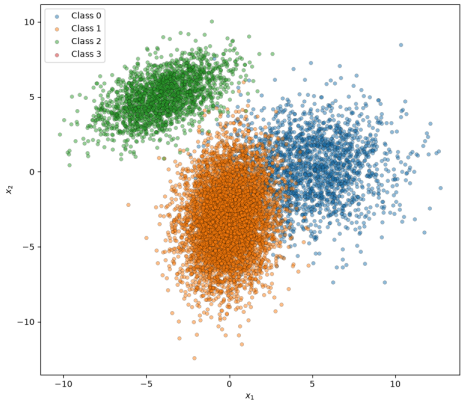


Figure 6: $\mathcal{D}^{100}$ Estimated Parameters Visualized at $N = 10000$

2.3.3 $\mathcal{D}_{1000}$

As visualized below in Figure 7, $\mathcal{D}_{1000}$ much more closely shows the the true distributions of the dataset. An optimal $K = 4.08$, closely mirroring the true parameters of the dataset. The average derived parameters and distribution is modeled below, sampled at $N = 10,000$.

$$\hat{\boldsymbol{\mu}}_0 = \begin{bmatrix} -0.03 \\ -4.50 \end{bmatrix}, \quad \hat{\boldsymbol{\mu}}_1 = \begin{bmatrix} 0.02 \\ 0.03 \end{bmatrix}, \quad \hat{\boldsymbol{\mu}}_2 = \begin{bmatrix} -4.02 \\ 4.99 \end{bmatrix}, \quad \hat{\boldsymbol{\mu}}_3 = \begin{bmatrix} 6.97 \\ -0.02 \end{bmatrix}$$

$$\hat{\Sigma}_0 = \begin{bmatrix} 1.99 & 0.45 \\ 0.45 & 2.00 \end{bmatrix}, \quad \hat{\Sigma}_1 = \begin{bmatrix} 2.48 & 0.92 \\ 0.92 & 2.40 \end{bmatrix}, \quad \hat{\Sigma}_2 = \begin{bmatrix} 3.98 & 1.51 \\ 1.51 & 2.02 \end{bmatrix}, \quad \hat{\Sigma}_3 = \begin{bmatrix} 2.04 & 1.14 \\ 1.14 & 5.06 \end{bmatrix}$$

$$\hat{\boldsymbol{\alpha}} = \begin{bmatrix} 0.40 & 0.30 & 0.20 & 0.10 \end{bmatrix}$$

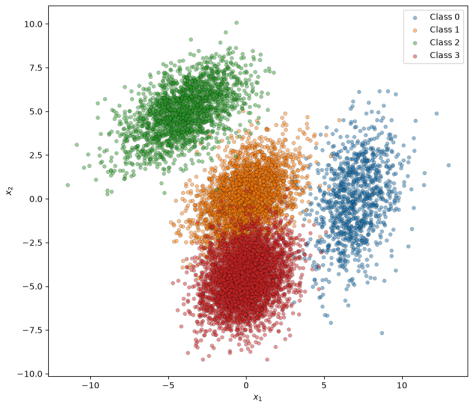


Figure 7: $\mathcal{D}^{1000}$ Estimated Parameters Visualized at $N = 10000$

2.4 Comparison

Based upon the results discussed in Section 2.3, the following table highlights the frequency that each of the given means were chosen for the dataset. As can be seen, the higher sample size of the given dataset leads to a higher correct frequency of correctly identifying the gaussian distributions of the dataset. Additionally, it is interesting to note that at a higher frequency of samples, the confusion regarding classes 0 and 1 meshing together is overall eliminated from this distribution, with $\mathcal{D}^{1000}$ having no estimated parameters below 4. These frequencies are further visualized in Figure 8.

Dataset	K=1	2	3	4	5	6	7	8	9	10
$\mathcal{D}_{10}$	0.97	0.03	0	0	0	0	0	0	0	0
$\mathcal{D}_{100}$	0	0.12	0.45	0.40	0.03	0	0	0	0	0
$\mathcal{D}_{1000}$	0	0	0	0.93	0.06	0.01	0	0	0	0

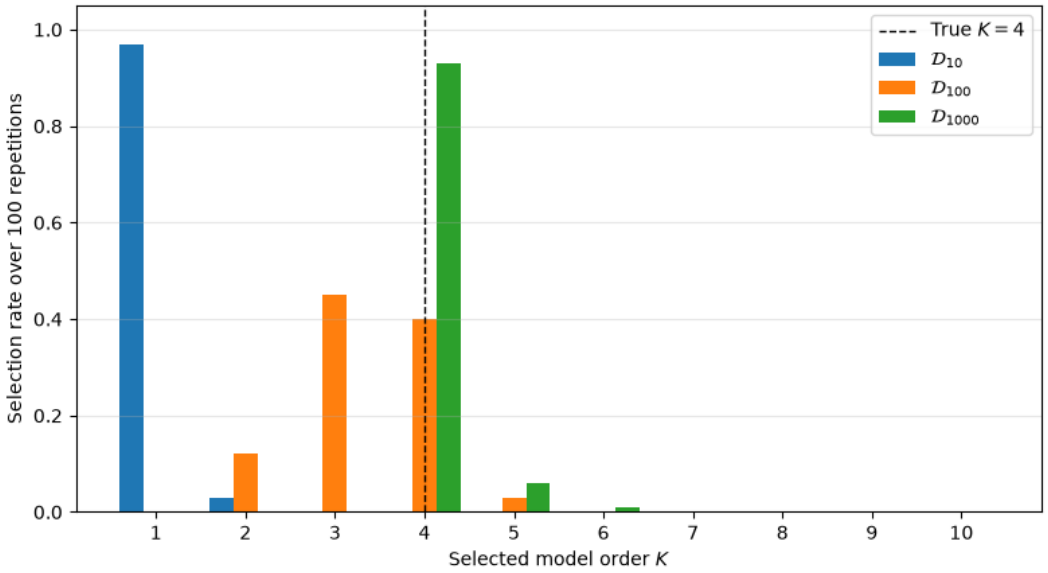


Figure 8: GMM Order Selection per Sample Size

Acknowledgment of AI Assistance

Claude (Anthropic), an AI assistant, was used extensively throughout this assignment. Claude explained the underlying theory, including MAP classification and its derivation under 0-1 loss, Mahalanobis distance and the Bhattacharyya bound, maximum likelihood training of MLPs with cross-entropy loss, K-fold cross-validation for model order selection, Gaussian mixture models, and the EM algorithm.

Claude reviewed and debugged my code, identifying issues including mislabeled samples in my data generation, label and loss-function mismatches in Keras, a network structure that did not match the assignment, a final model taken from a single cross-validation fold, slow training, an incorrect overlap distance in my Question 2 parameters, and several errors in my GMM model-selection experiment. Claude also wrote substantial portions of the code, including the MAP classifier, the cross-validation and random-restart training routines, the results plots, the repeated GMM experiment and selection-rate computation, and the alignment procedure used to average fitted parameters. It ran independent checks that confirmed my MAP error estimate, and suggested part of the Question 2 parameter choice. I integrated, ran, and adapted this code, and all reported numerical results were produced by running it.

In addition, Claude provided LaTeX formatting for equations, tables, and figures, drafted portions of the report text (including Sections 2.1 and 2.2 and the software-verification paragraph in Section 1.4), reviewed drafts against the assignment requirements, and drafted this acknowledgment. The transcript for this session can be found here: https://claude.ai/share/0639e8f7-0660-4bef-bc8f-2fd130eb4a8d

All code and figures can be found at: `https://github.com/RonanEngel/EC5644_Assignments`